PHISHGUARD

Phishing URL Detection System

A rule-based desktop application for identifying suspicious URL patterns

Project type	Personal cybersecurity portfolio project
Author	Ms. Saniya Sudhir Kadam
Technology	Python • Tkinter • Standard Library

Contents

- 1. Project Overview
- 2. Problem Statement
- 3. Objectives and Scope
- 4. Technologies and Requirements
- 5. System Design and Workflow
- 6. URL Analysis Methodology
- 7. Application Interface and Features
- 8. Input Validation and Error Handling
- 9. Testing and Observed Results
- 10. Limitations and Security Considerations
- 11. Future Enhancements
- 12. Conclusion
- Appendix A — Running the Application

1. Project Overview

PhishGuard is a beginner-friendly desktop application developed in Python to analyse URLs for common patterns that may be associated with phishing or deceptive links. It provides a graphical interface built with Tkinter, accepts a URL from the user, evaluates a set of predefined indicators, calculates a risk score, and displays a category with explanations.

The application is a rule-based educational tool. It does not connect to the submitted website, query a live threat-intelligence service, or use a machine-learning model. Its result is an assessment of URL characteristics only, not a definitive verdict about the website.

2. Problem Statement

Phishing messages often contain links designed to imitate trusted services or persuade users to share passwords, one-time passwords, payment information, or other sensitive details. A URL may contain warning signs such as misleading structure, unusual domain characteristics, link-shortening services, or words commonly used in deceptive messages. Users may overlook these clues when opening links.

The project addresses this awareness problem by providing a simple interface that highlights selected URL patterns and explains why they contributed to a risk score.

3. Objectives and Scope

Objectives:

- Accept a URL through a desktop graphical interface.
- Validate basic URL input and support domain-only input by adding an HTTPS scheme for parsing.
- Inspect predefined URL characteristics using Python rules.
- Assign weighted risk points to detected indicators and cap the total score at 100.
- Classify results as LOW RISK, SUSPICIOUS, or HIGH RISK.
- Display the reasons behind the score and provide general cautionary advice.
- Maintain a short, in-memory list of the five most recent scan summaries during the running session.

Scope:

PhishGuard analyses the URL string supplied by the user.

It does not inspect page content, redirect chains, certificates, DNS records, hosting reputation, or website behaviour.

4. Technologies and Requirements

Technology / module	Purpose in the project
Python	Main programming language used for the detection logic and application.
Tkinter	Builds the desktop graphical user interface.
ttk	Provides the progress bar and themed widget styling.
re	Imported for regular-expression support; no active use is present in the supplied source.
ipaddress	Checks whether the parsed hostname is an IP address.
urllib.parse	Parses URLs and decodes percent-encoded text.

System requirements

- Python 3 with Tkinter available.
- A desktop operating system supported by the installed Python/Tkinter environment.
- No third-party Python packages are required by the supplied source code.

5. System Design and Workflow

The application has two main parts: **a URL analysis function and a Tkinter interface.**

- The analyse_url(raw_url) function performs validation, extracts the hostname, evaluates indicators, calculates the score, and returns a result dictionary.
- The PhishGuardApp class manages user input, button actions, result presentation, and recent scan summaries.

Processing workflow

1. The user enters a URL or loads the built-in example.
2. The application calls analyse_url() when the Scan URL button is pressed or Enter is used.
3. The function validates the input and parses the URL.
4. Predefined rules are checked and weighted points are added when indicators are found.
5. The score is capped at 100 and mapped to a risk category.
6. The function returns the URL, hostname, score, category, advice, and detection reasons.
7. The GUI displays the result, updates the progress bar, and adds the summary to the recent-scan list.

6. URL Analysis Methodology

The detection engine uses a weighted rule-based approach. Each matching condition adds a fixed number of points. Multiple indicators can contribute to one result. The final score is limited to a maximum of 100.

URL indicator	Points assigned
URL does not use HTTPS	+8
@ symbol in the network location	+25
Hostname is a raw IP address	+30
Punycode marker (xn--) in hostname	+20
Known URL-shortening hostname	+15
Five or more hostname labels	+12
Three or more hyphens in hostname	+8
URL length greater than 100 characters	+8
URL length greater than 75 and up to 100 characters	+4
Hostname uses a listed higher-risk TLD	+12
Unusual port other than 80 or 443	+10
Invalid port number	+10
One suspicious keyword found	+5
Two or more suspicious keywords found	+9
Embedded username or password	+20

The configured shortening domains include bit.ly, tinyurl.com, t.co, goo.gl, ow.ly, is.gd, buff.ly, cutt.ly, rb.gy, and shorturl.at. The configured suspicious TLD list includes zip, mov, click, top, xyz, work, country, gq, tk, ml, cf, and ga. The suspicious-word list includes login, signin, verify, verification, update, secure, account, password, banking, wallet, payment, confirm, free, gift, bonus, and urgent.

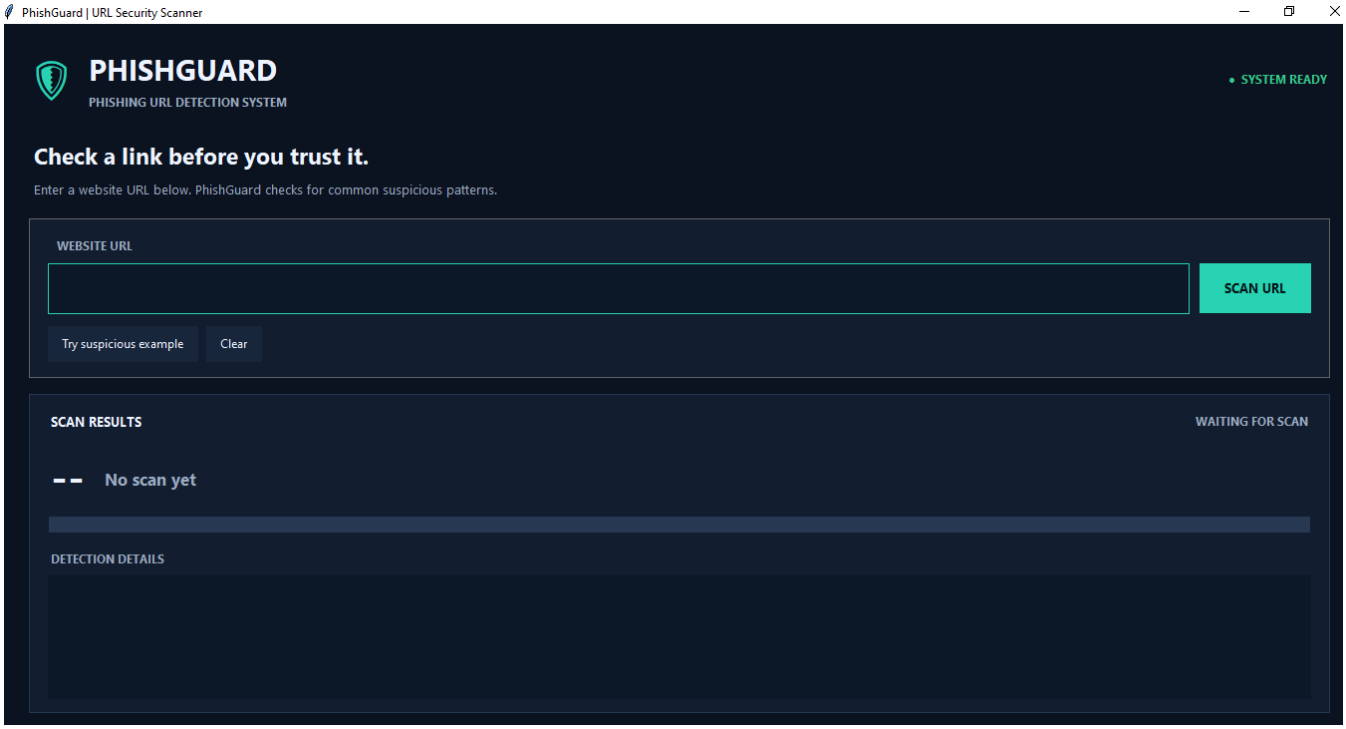
Risk classification

Score range	Category	Application guidance
0–17	LOW RISK	No major warning signs were found by these checks; this does not prove safety.
18–39	SUSPICIOUS	Be careful and verify the website address using its official source.
40–100	HIGH RISK	Avoid opening the link; do not enter passwords, OTPs, or payment details.

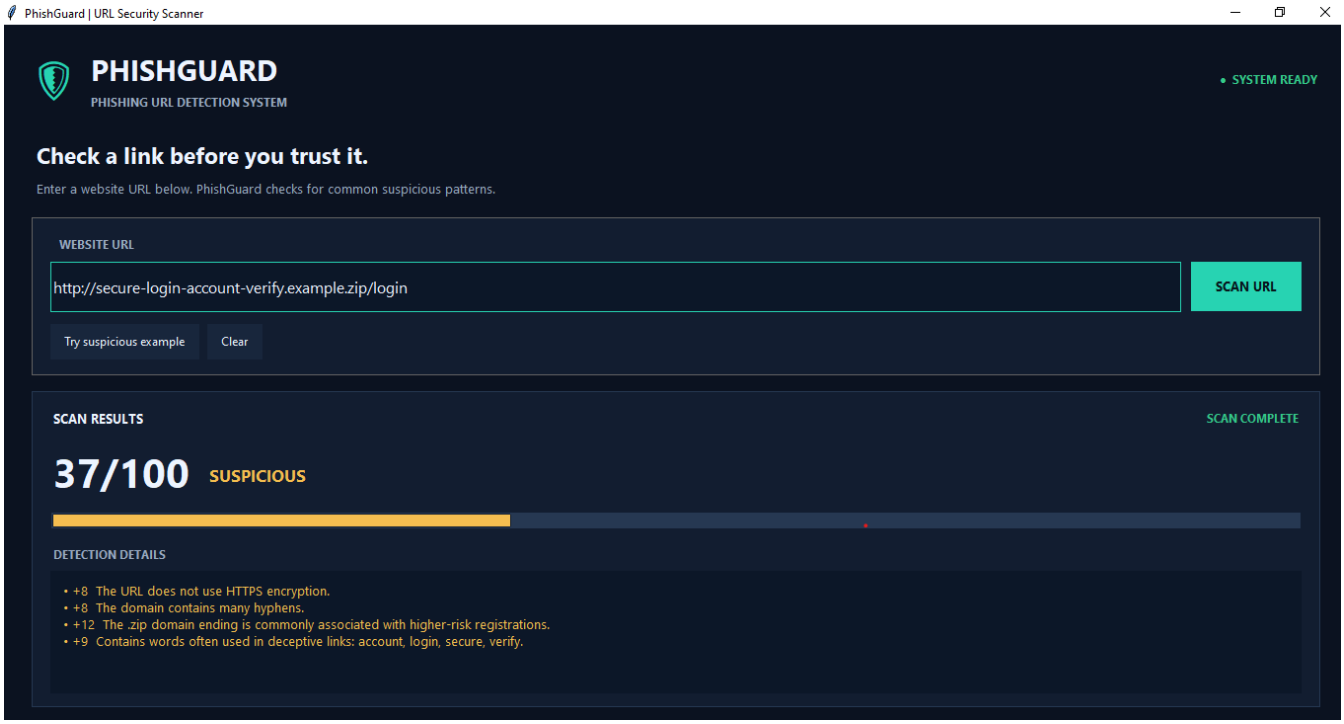
7. Application Interface and Features

- **Application header:** displays the PhishGuard name and a system-ready indicator.
- **URL input field:** accepts the website address to be analysed.
- **Scan URL button:** starts analysis; pressing Enter also triggers scanning.
- **Try suspicious example button:** loads a predefined example URL for demonstration.
- **Clear button:** resets the visible result and input field.
- **Scan results panel:** displays the numerical score, risk category, and a progress bar.
- **Detection details:** lists the reasons and points associated with detected patterns.
- **Recent scans:** shows up to five hostname/category summaries in memory while the application is running.
- **Advice label:** displays a cautionary message based on the resulting category.

The interface uses a dark colour palette with contrasting text and status colours. Green is used for low-risk presentation, yellow for suspicious results, and red for high-risk results.



Main window before scanning



Example scan result

8. Input Validation and Error Handling

- An empty input raises a validation message asking the user to enter a URL.
- Whitespace within the URL is rejected.
- Only HTTP and HTTPS schemes are accepted.
- A hostname must be present and must contain a dot.
- A scheme is prepended as HTTPS when the user enters a domain without one.
- Invalid port values are handled as a detection indicator when the URL parser raises a port error.

Validation errors are caught by the GUI and displayed in the detection-details area instead of terminating the application.

9. Testing and Observed Results

A preliminary local check was performed against the supplied source file. The file passed Python syntax compilation. The following function-level checks were executed; these are limited checks, not a complete security validation or comprehensive test suite.

Test input / condition	Observed result	Expected handling
https://example.com	0 — LOW RISK	Valid URL analysed; no configured warning patterns matched.
http://secure-login-account-verify.example.zip/login	37 — SUSPICIOUS	Multiple configured indicators contributed to the score.
Empty string	Validation error	Request an input URL.
URL containing whitespace	Validation error	Reject input containing spaces.

The example URL score is based on the supplied rules:

- HTTP scheme (+8)
- Multiple hyphens (+8)
- Listed .zip TLD (+12)
- Multiple suspicious words (+9)

Giving a total of 37.

10.Limitations and Security Considerations

- Pattern-based detection can produce false positives and false negatives.
- A low-risk result does not mean that a website is trustworthy or safe.
- The program does not open or fetch the URL and does not inspect website content.
- The program does not follow redirects or expand shortened links to reveal their final destination.
- The program does not use a live phishing database, threat-intelligence API, or machine-learning model.
- The recent-scan list is held in memory and is not saved to a file or database.
- The configured TLD, keyword, and shortening-service lists are fixed in the source code and may become outdated.

Users should verify important links through an organisation's official website or trusted communication channel. PhishGuard is an educational aid and should not be treated as a complete security product.

11. Future Enhancements

- Integrate a trusted threat-intelligence or phishing-reputation API.
- Add optional redirect inspection in a controlled and secure way.
- Improve domain analysis using additional carefully validated features.
- Store scan history locally with user consent.
- Add exportable scan reports.
- Create automated unit tests for individual detection rules and edge cases.
- Improve accessibility and add packaging for easier desktop installation.

12. Conclusion

PhishGuard demonstrates how Python can be used to build a desktop cybersecurity awareness tool. It combines URL parsing, IP-address validation, predefined pattern checks, weighted scoring, input validation, and a graphical interface. By presenting both a risk category and the reasons behind it, the application helps users understand selected warning signs in URLs. Its rule-based design makes the detection process explainable and suitable for learning and portfolio demonstration, while its limitations make clear that it is not a substitute for comprehensive website security analysis.